

TERNARY CNN LAYER

Lucas Erlacher, Maximilian Mosler, Robin Oester, Yuliia Sumovska

Department of Computer Science
ETH Zurich, Switzerland

ABSTRACT

Ternary Neural Networks (TNNs) represent a promising research direction in neural networks by utilizing ternary weights (+1, 0, -1) to achieve computational efficiency and adaptability for resource-constrained platforms. This paper introduces several optimization techniques aimed at reducing computational demand and improving data handling efficiency. The optimized implementation leverages SIMD operations, fusing of different algorithmic stages, cache blocking and other techniques to enhance the ternarization and packing of convolutional network tensors, bit-wise GEMM and activation function. The strategies presented in this work resulted in reduced computational complexity and an increase in execution speed. The speedup analysis shows that the proposed optimized algorithm achieves up to a 13.3% speedup over the baseline implementation for ternarization. Additional measurements were performed to analyze performance in typical scenarios for TNNs, specifically for using 3x3 and 1x1 kernel sizes. These measurements demonstrate up to 1.3x speedup for the 3x3 kernel size configuration, and up to 10x for 1x1 case.

1. INTRODUCTION

The rising demand for deploying deep learning models in resource-constrained environments (e.g. mobile phones) requires an advancement in a computational efficiency. A naive ternary convolution implementation served as the baseline and a starting point for our optimization road[1]. The naive version is structured to be simple and direct which makes it easy to understand at the cost of lacking performance. Our paper proposes an optimized version of the TNN layer which addresses several inefficiencies of the baseline version.

The algorithm consists of four main sections: data preparation (involving ternarization and img2row), bit-wise general matrix-matrix multiplication (GEMM) and Parametric Rectified Linear Unit (PReLU) as activation function. A diagram of the different stages of the end-to-end pipeline is provided in Figure 1.

We divided the baseline implementation into two phases: Ternarization/img2row and GEMM/PReLU. For the ternarize img2row part, we focused on reducing temporary data-

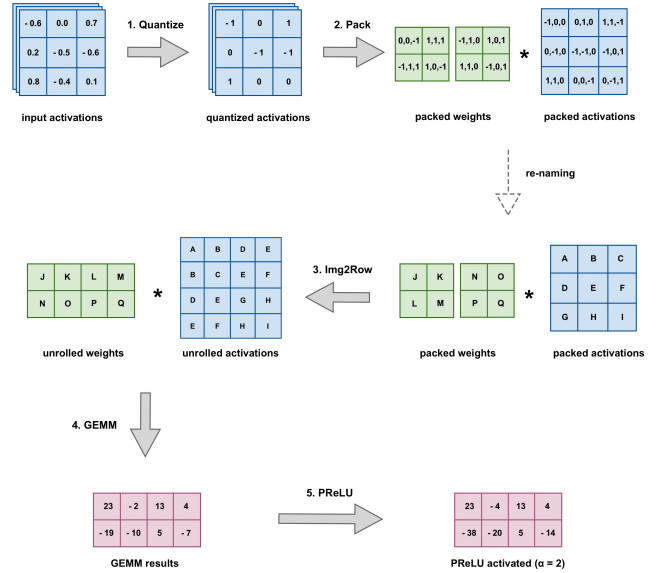


Fig. 1. Overview of the different stages of the ternary CNN algorithm.

writebacks, as the baseline used a buffer for redundant writebacks, increasing memory overhead. We also optimized the bit-packing process in the ternarize algorithm, which was computationally intensive. In the GEMM phase, the `popcnt` instruction was a bottleneck due to its low throughput (1 instruction/cycle), limiting the overall performance. We explored methods to bypass this limitation to improve performance. Additionally, we considered the use of multiple TNN layers in sequence, analyzing our algorithms' performance for 1x1 and 3x3 kernel sizes. We implemented the ternarize/img2row part for a separate data format and developed a novel approach for the 1x1 case.

2. BACKGROUND ON THE ALGORITHM

Ternary neural networks (TNNs) use low precision numbers to represent activation and weight tensors of CNNs. This

reduces storage and computation power considerably (compared to 32-bit floating point numbers) while still achieving good accuracy in selected benchmarks [1]. The algorithm for TNNs as proposed in this paper is depicted in Figure 1. It also forms the baseline approach to which we compare our methods. The algorithm proceeds in 4 stages: (i) the input tensor having shape $BN \times C \times H \times W$ (for batch size BN , number of channels C , height H , width W) and weight tensor $KN \times C \times KH \times KW$ (for KN being the number of kernels, KH and KW are the kernel dimensions) are quantized to ternary values $\{+1, 0, -1\}$ by thresholding along the channel dimension with parameters τ and $-\tau$. Then, these values are packed into 64-bit integers shrinking both tensors by a factor of 32 since each value can be represented as 2 bits. Additionally, they are reshaped such that the channel forms the last dimension of the tensor shape, i.e., the input tensor becomes $BN \times H \times W \times C_{pack}$ and the weights $KN \times KH \times KW \times C_{pack}$. We refer to this step as *Ternarization*. (ii) The *img2row* part prepares the packed input for the application of a bit-wise matrix-matrix multiplication. For this purpose, the input values are copied to several places in the unrolled input tensor (depending on padding p , stride s and the shape of the weight tensor). The resulting unrolled input tensor has shape $BN \times H_{out} \times W_{out} \times KH \times KW \times C_{pack}$. (iii) Due to this particular layout of data in memory, general matrix-matrix multiplication (GEMM) can be applied to compute the convolution result. In this case, we view the unrolled data tensor as a $M \times K$ matrix with $M = BN * H_{out} * W_{out}$, $K = KH * KW * C_{pack}$ and the weights as $N \times K$ matrix with $N = BN$. The memory layout of the data and the weights allows to compute GEMM as many bit-wise dot products of row vectors. This is similar to a matrix-matrix multiplication where the second matrix is in column-major order exhibiting good cache locality. The resulting tensor has shape $M \times N = BN \times H_{out} \times W_{out} \times C_{out}$ where $C_{out} = KN$. (iv) Parametric ReLU (PReLU) is used as a final step on all output values. PReLU is defined as $f(y) = \alpha^{1_{y < 0}} * y$ for $\alpha > 0$. Even though the input and weights are of type float, after quantization (which packs the values in 64-bit integers), only integer operations are performed. We define the cost measure C as the amount of relevant intops that are performed by the algorithm. This excludes comparisons, loop counters and memory operations. Thorough analysis shows $C_{quant} = BN * H * W * C$ the cost for performing quantization on the input tensor (analogous for the weights) under the assumption that the values are uniformly distributed into the 3 regions defined by τ . GEMM has a cost $C_{gemm} = M * N * K * 7 + M * N * 2$. Finally, the PReLU stage requires $C_{prelu} = M * N * 0.5$ intops assuming half of the computed values are negative. Summing these costs up results in $C_{total} = C_{quant} + C_{gemm} + C_{prelu}$ intops performed by the end-to-end TNN algorithm.

3. OPTIMIZATIONS PERFORMED

We now present the functions implemented as part of this project and the optimizations they contain.

3.1. Ternarization/Img2Row for NCHW Format

The functions presented in this section are intended to be used in the very first layer of a CNN as the input tensor to a CNN typically uses the NCHW (batch size, channel dimension, input image height, input image width) dimension-order. Since the baseline implementation always outputs NHWC tensors we decided to adhere to this ordering of dimensions for the outputs of our layers.

We have implemented and evaluated two different loop orderings for the NCHW-format-fusion of the ternarization and *Img2Row* stages: Ternarize loop order and *Img2Row* loop order. For each loop order we have implemented 3 versions which will be discussed in the following.

Img2Row Loop Order

The first approach we took to fuse ternarize-packing and *Img2Row* was to use the loop order of the *Img2Row* computation: N-OH-OW-KH-KW (batch size, output image height, output image width kernel height, kernel width). We hypothesized that this loop order would likely exhibit good write locality during the writeback of the packed values, as the tensor used for writeback follows the same dimension ordering. This would however come at the cost of recomputing certain values multiple times as there is now no temporary buffering of computed values.

The first version of our *Img2Row* loop order fusion is a functional fusion but largely unoptimized. It required us to derive formulas for the parameters h (input image height) and w (input image width) in terms of the *Img2Row* loop variables (oh , ow , kh and kw). In order to obtain a correct fusing testing the computed h and w on a set of simple conditions (non-negativity and bounds of the input image) is required. After the fusing was completed, the loop over variable ic exhibited a highly regular structure which we managed to exploit by replacing said loop by a conditional statement and (if necessary) the computation of the ic index through only one division operation.

Noticing that the first version was strongly compute bound (caused by the re-computation of packed values) we decided to vectorize the packing stage of the algorithm. We used AVX2 meaning that instead of one we were now able to ternarize and pack 8 float values in parallel. Due to the input format of this section (NCHW) we unfortunately have to load the values from non-contiguous places in memory. Furthermore, due to the fact that there can be an overhang in the channel dimension (i.e. it can be the case that the width of the channel dimension can't be packed into a multiple of 64 without a remainder) we have to ternarize and pack this

potential overhang in a non-vectorized way.

The third and final version of the `Img2Row` loop order fusing improves upon the second version by adding a number of optimizations to further increase computational efficiency. First, we derived formulas for tight loop bounds of `kh` and `kw` which allowed us to remove an `if` check inside of these loops. Furthermore, we unrolled the channel dimension loop by a factor of two which enabled us to remove the computation of `ic` and to make use of both `p1` and `p2` (i.e. both values that the quantization-packing function returns) in the same iteration (in the previous versions we had only written back either value in any given loop iteration).

Ternarize Loop Order

After preliminary evaluations, the `Img2Row` fusion approach failed to demonstrate a significant speedup over the unfused baseline implementation. Therefore, we set out to try a different approach to fusing `ternarize-pack` and `Img2Row`. We decided to implement this fusion using the `ternarize` loop order: `N-H-W` (batch size, input image height, input image width). This approach would allow to avoid computations at the cost of worse write locality in the writeback of the packed values.

In the first version of our implementation we focused on the fusing `Ternarize` and `Img2Row` algorithms (and did not yet employ other optimizations). This implementation aimed to remove intermediate writebacks of computed values and instead writes data directly to the final location in the `Img2Row` tensor. This fusion involved careful consideration of indices for writeback as well as deriving formulas for the output image dimensions (`OH` and `OW`) from the input image dimensions (`H` and `W`).

In preparation for the implementation of the second version we identified the most computationally expensive part of the algorithm to be bit-packing. Therefore it was decided to vectorize this section using `AVX2`. As in the `Img2Row` loop order fusing we had to load values non-contiguously and had to use non-vectorized code for the overhanging part of the channel dimension. In the third version we implemented additional optimizations to increase the compute efficiency of our fused code. Note that in the following paragraph we are assuming that the stride parameter is fixed to 1. To optimize the value-storing section, we eliminated the `if` condition wrapping the writeback-code by redefining the boundaries of the writeback loops. This adjustment ensures that the loops only process values within the correct bounds, automatically excluding any out-of-bound values that previously required an `if` condition check. Also, during this step we explored the option of utilizing a temporary buffer to store computed values before they are being written to memory. The buffer was intended to speed up the writeback process by accumulating computed values and afterwards writing them in large chunks. However, this implementation unfortunately

did not provide any runtime improvement. The conversion of the data format from `NCHW` to `NHWC` as the data flows through the ternary layer hindered the buffer's effectiveness, as each value still required an individual writeback to the correct position.

3.2. Ternarization/Img2Row for NHWC Format

As we have seen before, the output format of the TNN algorithm is a tensor of shape $BN \times H \times W \times C$. This format can be exploited for multi-layer CNNs, where the output of the previous layer is used as an input to the next one. Since packing is performed along the channel dimension, we expect good cache locality when packing the input tensor. Therefore, we built a separate version of `ternarization/img2row` for this data format. In total, we performed 4 major optimization steps: (1) Fusing of `ternarization` and `img2row` into one algorithm. (2) Apply SIMD optimizations to pack the input tensor. (3) Buffer along the channel dimension to copy C_{pack} values to the unrolled input tensor. (4) Make use of a larger buffer to copy up to $KW * C_{pack}$ values at the same time to the unrolled input tensor. This method additionally assumes $s = 1$.

In order to compare our approaches to the baseline, we reshape the input tensor from $BN \times H \times W \times C$ to $BN \times C \times H \times W$ before feeding this tensor to the baseline algorithm described above. By not measuring the runtime for this operation, we ensured fairness in performance comparisons. As a first step, we developed version (1) that is concerned with fusing `ternarization` and `img2row`. By looping over the input tensor, the values are packed along the channel. After each packing (of at most 64 values), the two 64-bit integers are copied to the correct locations in the unrolled tensor. This optimization on its own, which removes the necessity of intermediate buffering, didn't result in a significant lower runtime for some basic measurements.

In version (2), we applied SIMD optimizations by using `AVX2` instructions to pack the channel values for each pixel. This allows us to perform the packing part up to 8 times faster as we can perform all computations on 256-bit registers, which holds 8 32-bit floats. Since the tensor format is `NHWC`, the channel values can be loaded from contiguous memory locations, thus, exploiting the good cache locality. Our hypothesis is supported an approx. 3.5-fold lower runtime. After gaining this significant speedup, we shifted gears and focused on the `img2row` part of this fused algorithm.

As opposed to the previous version, (3) stores all C_{pack} channel values of a given pixel in an intermediate buffer. We observe that in the unrolled input tensor, all channel values also appear consecutively. Therefore, we can use `memcpy` to copy this buffer to the correct locations. Running experiments with a larger number of channels (e.g. 256) also

achieved a better performance - likely due to the better write locality compared to the previous version.

In (4), we take this idea of copying larger chunks one step further. Figure 1 shows that up to KW consecutive values in the input appear together in the unrolled tensor, whenever $s = 1$. We exploit this observation by blocking the H and W loops with the blocking factors BH and BW . The value buffer then has a size of $BH * BW * C_{pack}$ values. These blocking sizes BH and BW form a tradeoff between copying more chunks of contiguous memory and time "wasted" to fill the buffer in which no memory copying is performed. After experimenting with different combinations, we found out that $BH = KH$ and $BW = W$ are suitable blocking values for our 3x3 kernel cases. This leads to a buffer size γ_B of $\gamma_B = BH * BW * C_{pack} * 2 * 8 = 48W$ bytes meaning that we are within the range of L1 cache size for smaller input sizes. Some initial results suggest that copying these larger chunks of memory is superior to all other methods since also write locality is exploited to a good extend. We further optimized for a specialized 3x3 kernel size version by unrolling the loops, using SSA and do some small-scale optimizations that are applicable when having this static kernel size. However, it resulted in similar performance compared to the general case.

3.3. GEMM/PreLU

First, during the GEMM part of the algorithm, the matrix dot product is calculated between the transformed tensor and the weight tensor, resulting in the convolution's feature map output. To optimize this, the baseline implementation performs a ternary bit-wise dot product using Gated-XOR logic, essentially using one XOR operation followed by two AND operations. The number of nonzero bits in the results of the AND operations is then counted using the popcount instruction.

Given this optimization of the GEMM computation, our initial idea was to enhance data locality. The baseline iterates over the input matrix in a row-major fashion, so we experimented with blocking the algorithm into square blocks of various sizes. However, this did not impact the runtime or performance of GEMM, likely because the baseline already effectively utilizes locality. Blocking would only be beneficial if the cached blocks were reused frequently, which is not the case for common kernel sizes of 3x3 and 1x1.

Using Intel VTune and Advisor to identify further bottlenecks, we found that GEMM is predominantly compute-bound. Therefore, we attempted to vectorize the blocked implementation using SIMD. Since Intel AVX2 does not provide a vectorized version of `popcnt` and our system lacks AVX512 capability, our vectorized computation only included XOR and AND operations, with `popcnt` computed in a scalar manner at the end of each iteration after storing the SIMD results. The runtime remained unchanged,

and VTune analysis revealed that the `popcnt` instruction was the dominant factor for both scalar and vectorized code versions. Our CPU's throughput for this instruction is only 1 on Port 1, and we use two `popcnt`s in the scalar and four in the vectorized version per inner loop iteration.

Resource contention on Port 1 is the main bottleneck, so we sought alternatives for the `popcnt` instruction. One alternative, a library from another paper [2], converts `popcnt` on AVX2 systems into a lookup table operation, provided the input is large enough. Replacing our non-vectorized popcounts with this library in the SIMD version, we found the implementation became much slower. Similar to the blocked version, kernel size was the issue, as most kernels are small, resulting in insufficiently large computations per inner loop iteration.

To address this, we tried using both the default `popcnt64` instruction and the new vectorized version. We unrolled the inner loop by a factor of four, performing two SIMD computations and splitting them between the regular `popcnt` instruction and the AVX2 library version. This approach worked for larger cases of N and K but was generally worse than other versions due to overhead and previous issues for very large N and K .

Our final approach to GEMM involved iterating over the smaller dimension (M or N) of the input in the inner loop to cache the second most inner loop data, providing temporal locality. This consistently outperformed the baseline.

Next, we focused on the PreLU activation function, initially attempting to fuse it with GEMM. However, PreLU uses control flow to determine which result to write, and incorporating this into GEMM led to a noticeable performance decline due to poor speculation. When GEMM executes the fused PreLU control flow, the pipeline performs branch speculation. Since the branch condition is based on a simple result value without patterns, predictions often fail, causing all speculatively executed instructions waiting in the reorder buffer at the commit stage to be squashed. We confirmed this after eliminating the suspicion of memory aliasing using the `restrict` compiler instruction and verifying with VTune to measure the impact of bad speculation.

3.4. End-to-End Algorithms

After splitting the TNN algorithm in these 2 parts and optimizing them on their own, we are set to create our own versions of end-to-end TNN. We considered two main cases: (i) General TNN. In this scenario, we apply a combination of our algorithms to arbitrary kernel sizes. The only constraint is $s = 1$, which allows us to use the large buffer version for the NHWC format. (ii) Specialized 1x1 TNN. In this case, $H = W = KW = KH = 1$, which enables case-specific optimizations. We implemented 2 specialized approaches for this scenario.

In both cases, our implementations are optimized for multi-layer executions. The main idea behind general TNN is the use of our variants for the different parts of the algorithm. The baseline is formed by using the default baseline implementation for the first layer. All subsequent layers perform a reshaping to NCHW, before applying the default implementation again. Our version uses the latest NCHW algorithm for the first and the latest NHWC algorithm for the remaining layers to perform ternarization/img2row. For GEMM/PReLU, we use the baseline approach.

If the TNN algorithm is used to compute the result of a 1x1 convolution with the additional constraints described above, we observe in particular that there is no need for img2row. The purpose of such a computation is to compute the result of a ternarized fully-connected MLP layer. As opposed to the general case, where in practice the problem size grows with increasing H and W in the input, here it increases with C and KN , which depend on the input and the weights. Therefore, we often have a larger weight tensor (if $KN > BN$) than input tensor. For better understanding, we can also visualize the computation as a fully-connected MLP with C_{pack} inputs and KN outputs, which is computed for a batch of size BN . This observations together with the knowledge we gained from the general case guided the optimization route for our specialized 1x1 algorithms, where we developed 2 solutions: (1) End-to-end fusing with input buffer: This version performs the whole TNN algorithm in one single function. It stores the quantized input tensor in a buffer before computing GEMM by iterating over all kernels. (2) SIMD popcount: This version is essentially the same as (1). However, for computing GEMM, we use a AVX2 popcount library and the default 64-bit `popcnt` instruction.

As stated in the summary, version (1) first packs the whole input tensor to an intermediate buffer using SIMD (see Section 3.2). This allows the reuse of the input tensor at a later stage with minimal memory-overhead. Alternatively, we could also buffer the weight tensor but since it is likely to be larger for the relevant cases (see our analysis above), we chose not to. After buffering, we iterate over all kernels and also use a buffer to store the packed channels of the weights. Then, we compute GEMM as in the baseline using the values from the two buffers with subsequent application of PReLU. We recall the GEMM computations, which performs bit-wise matrix-matrix multiplication.

The innermost loop iterating over $K = KH * KW * C_{pack} = C_{pack}$ performs much more computations than in the general TNN case. Since we already identified the `popcnt` instruction to be the bottleneck (having no SIMD equivalent), we used a library[2] to compute many `popcnt` instructions in parallel through equivalent AVX2 code in version (2). Our initial results were unsatisfying as runtime was worse than in the default setting. We then implemented a more sophisticated approach that uses 64-bit `popcnt`

and the AVX2 library to maximize the throughput of `popcnt`. For this purpose, we blocked the K loop by a factor of $2 * KB$ and unrolled the innermost loop by a factor of 8. This allowed us to calculate intermediate values (before `popcounting`) for 4 values and to store these results in a separate buffer. This buffer is later used to perform `popcounting` on a much larger array by the library. The other 4 values are simply processed as before by using non-vectorized code and the 64-bit `popcnt` instruction. Our hypothesis was that we can maximize port utilization with the machine performing `popcnt` in parallel to preprocessing the other values for an efficient large-scale `popcount` (of size KB) later on. This procedure also involves tweaking KB , which we set to $KB = 256$ after some experiments.

4. EXPERIMENTAL RESULTS

This section is dedicated to the experiments we performed on our algorithms in comparison to the baseline. We first discuss the results for fusing ternarization with img2row before showing the performance of our GEMM/PReLU implementations.

Experimental setup.

The experimental setup was conducted on an Arch Linux machine with an Intel i7-10510U @ 4.2 GHz Comet Lake processor featuring a 32KB L1 cache, 256KB L2 cache, and 8MB L3 cache, alongside 16GB of RAM. We utilized the g++ compiler with optimization flags `"-std=c++11 -msse4.1 -msse4.2 -mavx2 -mpopcnt -march=core-avx2 -O3"`. Please note that the roofline plots were done at 1.8 GHz not 4.2 GHz.

For comparison purposes, we focused on two types of CNN layers. The first one consists of 3x3 kernels. In practice, this is the most prevalent kernel size and widely deployed in several applications. The second test case uses a 1x1 kernel. This allows us to measure the performance of a fully-connected ternarized MLP layer. Both types use an incremental variable n that is used to measure at scale. The parameters used for our experiments are the following:

- 3x3 case: $BN = 1, C = 64, KN = 16, KH = KW = 3, p = 1, s = 1, W = H = n$
- 1x1 case: $BN = 32, H = W = KH = KW = 1, p = 0, s = 1, C = KN = n$

Both test cases are designed to have the same input and output dimensions. Hence, they can also be used for multi-layer measurements. If not stated otherwise, we executed the test cases on warm caches and took the median of five experiments. Each experiment executed the algorithm until the runtime was above 10^9 cycles and took the mean execution time to mitigate external effects.

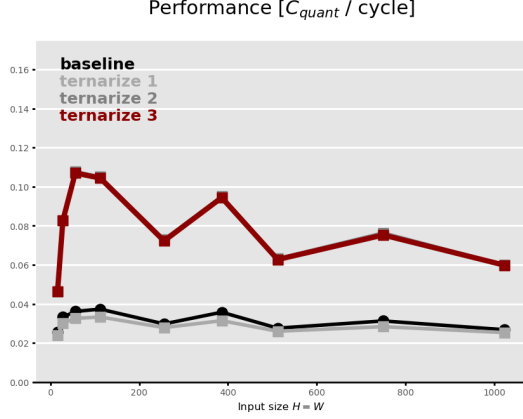


Fig. 2. Performance of the ternarize loop order implementation of fusion of ternarization and img2row. We omitted version 2 since it performs almost identical to is ternarize 3.

Results. We first discuss the results of measuring the NCHW-to-NHWC functions. As discussed we have implemented two versions of the fusion of ternarize-pack and im2row: im2row loop order (N-OH-OW-KH-KW) and ternarize loop order (N-H-W).

We first discuss the performance of the fusing in ternarize loop order presented in Figure 2. The first version of the fusion in ternarize loop order does not manage to beat the baseline. The second and third versions are able to clearly outperform the baseline: "Ternarize 3" for instance achieves a speedup of 2.2 over the baseline in the largest (input image side length = 1024) test case. The reason for this improved runtime (in comparison to the fusion implementations in im2row loop order) is likely the fact that the ternarize loop order does not incur any recomputations of the packed-values. The reduced writeback locality of the ternarize-loop order approach does not seem to matter as much as avoiding re-computations. As expected, the compute optimizations applied to version 2 and 3 of the ternarize loop order fusing improve the runtime over version 1. When inspecting the performance plot given in Figure 2 we also observe a clear advantage of ternarize 2 and 3 over ternarize 1 in terms of performance: in the largest test case (1024) our best versions exhibit a 2.2x performance increase over the baseline.

Roofline for img2row loop order of NCHW format.

In this section we will discuss observations received from roofline plot for Im2Row loop order of NCHW format which includes all 3 versions of our implementation (Figure 3). The first version of this loop order is computationally bounded as indicated by its position in the compute bound region. The second version is located higher in the compute bound region compared to the first version which indicates improved performance due to utilization of SIMD instructions. The

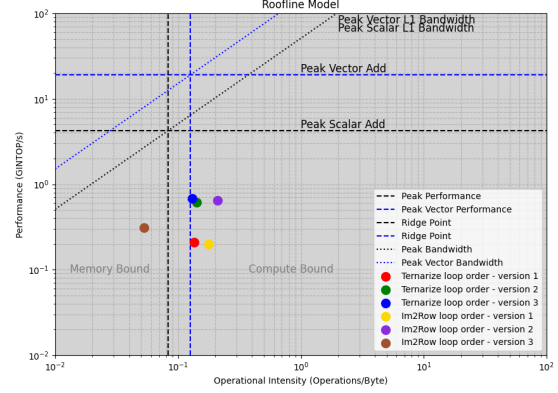


Fig. 3. Roofline plot for Im2Row and Ternarize loop orders of NCHW format.

third version is located in the memory bound region due to the removal of conditional statement and redundant computations associated with it. This results into reduced operational intensity leading to this memory bound behaviour.

Roofline for Ternarize loop order of NCHW format.

Figure 3 demonstrates roofline plot for Ternarize loop order of NCHW format. In the first version of our implementation we observed that our algorithm is located in the compute bound region and has a relatively low performance. Conversely, the second version demonstrated an improved performance which aligns with the usage of AVX2 instructions. This version reflects the enhanced computational efficiency of the algorithm achieved through vectorization. Furthermore, the roofline plot demonstrated that the third version did not exhibit any significant improvements. It demonstrated only slightly improved operational intensity which does not have a significant impact on the algorithm's overall performance. By comparing rooflines of both loop orders we came to the conclusion that the Ternarize loop order provides higher performance resulting in improved overall efficiency of the algorithm.

Ternarization/img2row fusing for NHWC format.

This experiment compares different versions of the fused algorithm that performs ternarization and img2row for TNNs. In total, we have 6 versions: baseline implementation (which is unfused), fused (see Section 3.2, algorithm 1), fused with SIMD packing (2), buffer along channel dimension (3), large buffer (4) and a specialized 3x3 version. The results are shown in Figure 4. To make the plot more readable, we omitted the specialized version which performed similar to the large buffer case. We also removed (3) from the plot, which performed worse than SIMD packing. This is due to the fact that $C = 64$ and therefore $C_{pack} = 1$, which makes channel buffering superfluous. However, for measurements with more channels, this optimization achieved

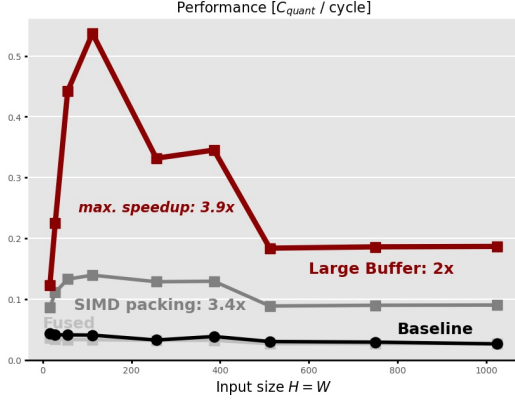


Fig. 4. Performance for fusing ternarization with img2row using the NHWC format.

significant speedups (e.g. for 512 channels and 3x3 kernel, it is around 1.5x faster than SIMD packing). We observe a total speedup compared to the baseline of up to 13.3x (!) for the large buffer version. A possible explanation for the huge performance benefit at $n = 112$ could be that the unrolled input tensor requires 1.8MB and can therefore be fully stored in L3. This makes the large buffer version most effective. However, thorough analysis must be done to confirm this hypothesis.

Multi-layer end-to-end. We investigated the same two scenarios as described in the beginning of the chapters. The only difference is that we performed end-to-end measurements for 3 layers. Looking at the performance of our best performing algorithm (that consists of our optimized parts), we observe a speedup of around 1.3x, which is devoted to our optimization in ternarization/img2row fusing. Since the baseline algorithm is already fast for GEMM/PReLU and runtime is dominated by this part (around 77.6% is spent doing these last two steps of TNN), the overall speedup is limited.

Looking at the performance of our two specialized 1x1 algorithms, we observe significant speedup numbers - especially at scale (see Figure 5). Interestingly, version (1), is faster than (2) until the channel dimension exceeds 9000. At this point, $C_{pack} = 141$ meaning that the 2 big popcounts done by the library are performed on buffer sizes of 544 bytes. This result coincides with the results from the paper [2], which claims that the AVX2 version is faster for arrays of size ≥ 512 bytes compared to 64-bit `popcnt`.

Lastly, for GEMM performance we measure the popcounts per cycle in 6, since it is computationally bound by them. As previously discussed our Opt function is the only function that actually improves upon the baseline for cases larger $N = K = 500$. The dynamic memory layout is likely responsible for this, as our test cases scale only N, K while keeping $M = 32$ small, meaning that the baseline has worse

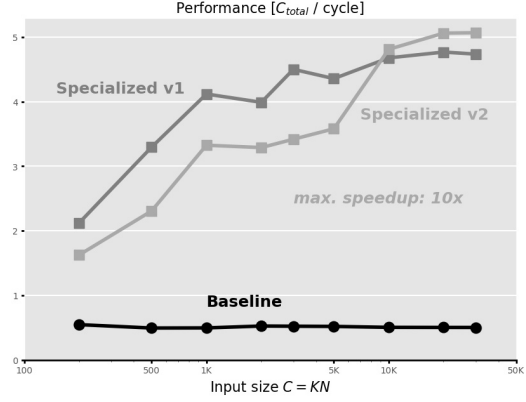


Fig. 5. Performance of our specialized 1x1 kernel size TNN algorithms.

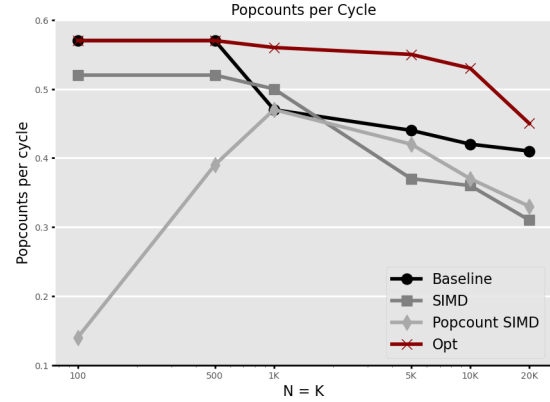


Fig. 6. Different GEMM optimizations in relation to the popcount bottleneck computation

locality for larger cases. Thus, we improve by up to 25% for cases in range $1000 < N = K < 10000$.

5. CONCLUSIONS

Our investigation into optimizing ternary CNNs focused on enhancing performance through various techniques. For NCHW-fusion, we found that the ternarize loop order (N-H-W) was more effective than img2row, and employing SIMD instructions for packing resulted in a 2.2x speedup. In multi-layer scenarios, using ternarization/img2row fusion in NHWC format and increasing contiguous data copying led to speedups of up to 13x. Adjustments in TNNGEMM_opt loop order significantly improved performance, though the greatest gains were seen with fused algorithms for 1x1 kernels, achieving up to 10x speedup. Our optimizations notably enhance TNN performance, especially in resource-constrained settings, aligning with state-of-the-art research on population counting in GEMM.

6. CONTRIBUTIONS OF TEAM MEMBERS (MANDATORY)

Lucas. Implementation of measurement code (especially the multilayer measurement code), Implementation and optimization of NCHW fusions (both Im2Row and Ternarize loop order versions): deriving (and debugging) the fusions, SIMD for bit-packing, unrolling of suitable loops and numerous other optimizations;

Robin. Implementation of blocking & SIMD versions for GEMM. Computations to perform fusing of ternarization/img2row. SIMD packing. Implementation of NHWC versions. Implementation of specialized 1x1 versions. Definition of cost measure. Performance plots and theoretical analysis of end-to-end algorithms & NHWC versions. Code for performance measurements.

Yuliia. Implementation and optimization of NCHW format: debugging correct indices for Im2Row and Ternarize loop orders, deriving correct boundaries for removal of conditional statement, implementation of buffer and its analysis, other minor optimizations of Ternarize & Im2Row part of the algorithm.

Max. Implementation of optimizations for GEMM and PreLU, specifically focusing on the popcount bottleneck and PreLU fusing. Popcount accumulation and popcount AVX2 library usage. Rooflines for all parts of the implementation as well as optimization insights with tooling and code instrumentation.

7. REFERENCES

- [1] Shien Zhu, Luan H. K. Duong, and Weichen Liu, “Tab: Unified and optimized ternary, binary, and mixed-precision neural network inference on the edge,” *ACM Trans. Embed. Comput. Syst.*, vol. 21, no. 5, oct 2022.
- [2] Wojciech Muła, Nathan Kurz, and Daniel Lemire, “Faster population counts using avx2 instructions,” *The Computer Journal*, vol. 61, no. 1, pp. 111–120, May 2017.